Number: P3068R2
Date: 2024-05-21
Audience: EWG & LEWG
Reply-to: Hana Dusíková

Table of contents

Allowing exception throwing in constant-evaluation

Introduction

Since adding the `constexpr` keyword in C++11, WG21 has gradually expanded the scope of language features for use in constant-evaluated code. At first, users couldn't even use `if`, `else`, or loops. C++14 added support for them. C++17 added support for constexpr lambdas. C++20 finally added the ability to use allocation, `std::vector` and `std::string`. These improvements have been widely appreciated, and lead to simpler code that doesn't need to work around differences between normal and constexpr C++.

The last major language feature from C++ still not present in constexpr code is the ability to throw an exception. This absence forces library authors to use more intrusive error reporting mechanisms. One example would be the use of `std::expected` and `std::optional`. Another one is the complete omission of error handling. This leaves users with long and confusing errors generated by the compiler.

Throwing exceptions in constant evaluated code is the preferred way of error reporting in the proposal adding *Static Reflection for C++26*. Some meta-functions can fail, and allowing them to throw will significantly simplify reflection code.

Changes

- R1 → R2: Added library wording, new examples, implementation description, and updated impact of changes.
- R0 → R1: Changed wording after consultation with Jens, added ex-